



UNIVERSITÀ DEGLI STUDI DI CATANIA
DIPARTIMENTO DI MATEMATICA E INFORMATICA
CORSO DI LAUREA MAGISTRALE IN INFORMATICA

Calvagno Fabio Santi

Smart University

RELAZIONE PROGETTO INGEGNERIA DEI
SISTEMI DISTRIBUITI

Anno Accademico 2025-2026

Sommario

Introduzione	2
1. Modello Architetture Generalizzato	3
1.1. Requisiti di Sistema e Obiettivi Progettuali	3
1.2. Il Modello N-Tier a Microservizi	3
1.3. Stack Tecnologico ed Ecosistema di Sviluppo	4
2. Architettura di Dettaglio dei Microservizi	5
2.1. Server di Autenticazione (AuthService)	5
2.2. Server Dati e Logica di Dominio: StudentService	6
2.3. Server Dati e Logica di Verbalizzazione: ExamService	8
2.4. Server Web e Presentation Layer: web-portal	9
3. Interfaccia Utente e Descrizione dei Casi d'Uso	10
3.1. Autenticazione e Identificazione Utente	10
3.2. Dashboard Amministratore	11
3.3. Dashboard Docente	12
3.4. Portale Studente	13
4. Trattazione Formale dei Design Pattern Utilizzati	14
5. Limiti Architetture e Trade-Off	15
6. Conclusioni e Sviluppi Futuri	16
Appendice A: Manuale Operativo e di Deploy	17
Requisiti di Ambiente ed Installazione	17
Guida all'Avvio Sequenziale dei Moduli	17

Introduzione

La continua evoluzione dei sistemi software enterprise ha evidenziato i severi limiti strutturali e operativi delle architetture monolitiche tradizionali. Nei sistemi monolitici, la logica di presentazione, le regole di business e le routine di accesso ai dati risiedono all'interno di un unico artefatto software. Questa forte coesione interna genera accoppiamenti indesiderati, rendendo estremamente complessa la manutenzione del codice, precludendo la possibilità di riscrivere in modo selettivo i singoli moduli applicativi e introducendo un punto singolo di fallimento (Single Point of Failure). Nel contesto specifico della gestione dei servizi universitari — caratterizzato da carichi di lavoro asimmetrici, in cui la fase di verbalizzazione degli esami presenta picchi temporali di traffico notevolmente superiori rispetto alle normali operazioni amministrative — l'adozione di un'architettura distribuita disaccoppiata risulta essere un requisito fondamentale.

1. Modello Architetture Generalizzato

1.1. Requisiti di Sistema e Obiettivi Progettuali

Il progetto Smart University nasce con l'obiettivo teorico e pratico di ingegnerizzare una piattaforma per la gestione delle carriere universitarie conforme ai più rigorosi principi di progettazione dei sistemi distribuiti. I requisiti fondamentali definiti per l'infrastruttura comprendono:

- **Isolamento dei Dati e Information Hiding:** Ogni microservizio deve possedere l'esclusiva titolarità sul proprio dominio di persistenza, vietando accessi diretti al database da parte di moduli esterni (Database-per-Service).
- **Sicurezza Autonoma e Controllo degli Accessi:** Il sistema deve garantire un'autenticazione centralizzata e robusta, rilasciando credenziali stateless verificabili in modo distribuito senza sovraccaricare il server di autenticazione (JWT Token & Reference Monitor).
- **Disaccoppiamento Temporale ed Asincronismo:** La comunicazione tra i microservizi per operazioni non bloccanti (quali la verbalizzazione e notifica del voto) deve avvenire in maniera asincrona mediante un broker di messaggi (RabbitMQ), garantendo che eventuali picchi di carico non impattino sulle prestazioni del client.
- **Tolleranza ai Guasti e Resilienza:** In presenza di malfunzionamenti o indisponibilità temporanea di uno o più microservizi backend, il Presentation Layer deve degradare le proprie funzionalità in modo controllato senza interrompere la navigazione dell'utente. Per consentire l'intervento tempestivo del Circuit Breaker, il client HTTP RestTemplate nel portal web è stato configurato con timeout espliciti di connessione e lettura socket pari a 3000 ms (3 secondi). In questo modo, un rallentamento o blocco di I/O su un microservizio backend attiva immediatamente la routine di Fallback senza congelare i thread del Web Server.
- **Prestazioni ed Ottimizzazione delle Risorse:** L'accesso ai dati ad alta frequenza di lettura (es. la media e lo storico del libretto studente) deve essere velocizzato tramite meccanismi di caching trasparente sviluppati secondo il paradigma della Programmazione Orientata agli Aspetti (AspectJ AOP).

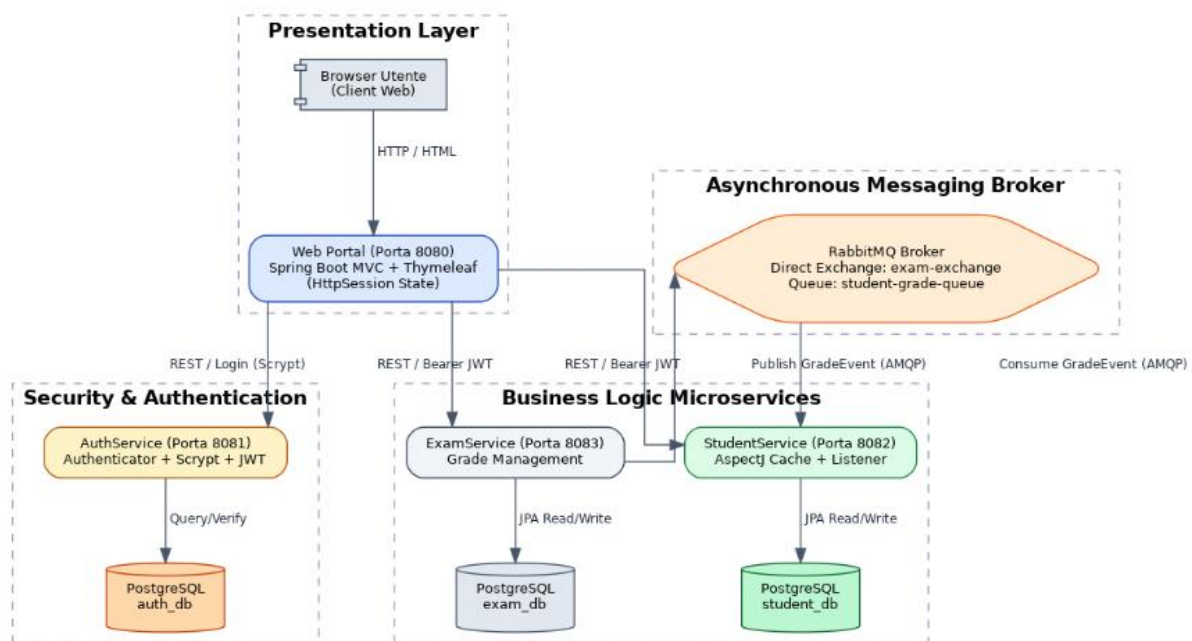
1.2. Il Modello N-Tier a Microservizi

Per soddisfare i requisiti di disaccoppiamento, manutenibilità e isolamento dei dati richiesti dalle moderne applicazioni distribuite, l'architettura della piattaforma *Smart University* adotta una rigorosa suddivisione in tre livelli logici (Modello N-Tier), distribuita strutturalmente su quattro moduli software indipendenti. Come illustrato nello schema generale di **Figura 1.1**, il sistema si articola nelle seguenti componenti:

- **Presentation Layer (Server Web - web-portal):** Sviluppato mediante il framework Spring Boot MVC e il motore di template Thymeleaf (in esecuzione sulla porta HTTP 8080), rappresenta il punto di ingresso per gli utenti. Gestisce le viste HTML dinamiche e adattive in base al ruolo del soggetto autenticato e orchestra le chiamate REST verso i microservizi di backend.
- **Business Logic Layer (Microservizi di Dominio):** Comprende tre microservizi disaccoppiati in esecuzione su istanze distinte: l'AuthService (porta 8081), responsabile dell'autenticazione e della sicurezza centrale; lo StudentService (porta 8082), dedicato alla

gestione delle anagrafiche e delle carriere degli studenti; e l'ExamService (porta 8083), preposto alla gestione degli appelli e alla verbalizzazione degli esami. La comunicazione tra questi componenti avviene sia in modalità sincrona via REST sia in modalità asincrona tramite il Message Broker RabbitMQ.

- **Data Access Layer (Basi di Dati Isolate):** In conformità con il pattern architeturale *Database-per-Service*, ciascun microservizio dispone di un'istanza PostgreSQL privata e isolata (auth_db, student_db e exam_db). Questo impedisce l'accesso diretto ai dati da parte di servizi terzi e soddisfa appieno il principio dell'Information Hiding.



1.1: Architettura generalizzata N-Tier a Microservizi di Smart University

1.3. Stack Tecnologico ed Ecosistema di Sviluppo

La scelta delle tecnologie componenti è stata guidata dalla necessità di utilizzare strumenti standard industriali dotati di elevata affidabilità. Il framework Spring Boot 3.x (Java 21) è stato impiegato per la costruzione rapida di microservizi RESTful autosufficienti con server Tomcat embedded. La sicurezza è garantita dalle librerie Auth0 java-jwt e dalla suite di cifratura Spring Security Crypto per l'algoritmo Scrypt. La messaggistica asincrona si avvale di RabbitMQ Broker mediante il protocollo AMQP 0-9-1. La resilienza sul client web è gestita da Resilience4j combinato con la libreria Vavr (Try.ofSupplier), mentre la messaggistica asincrona per la verbalizzazione si avvale dell'AMQP Broker RabbitMQ 3.12 (con Direct Exchange 'exam-exchange' e coda 'student-grade-queue').

2. Architettura di Dettaglio dei Microservizi

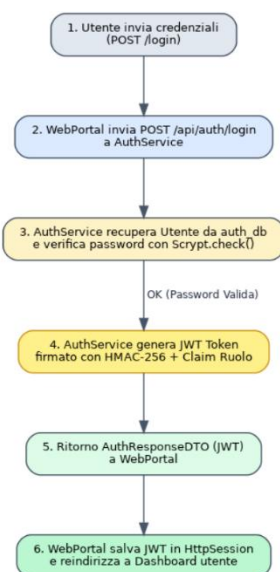
Ciascun microservizio componente la piattaforma Smart University è stato progettato come un modulo autonomo e disaccoppiato, dotato di un proprio ciclo di build, configurazione e persistenza. Di seguito viene presentata l'analisi dettagliata dei singoli componenti backend e frontend, evidenziando le scelte implementative e la corrispondenza con i pattern formali della letteratura.

2.1. Server di Autenticazione (AuthService)

Il microservizio AuthService (in esecuzione sulla porta HTTP 8081) ricopre il ruolo centrale di autorità di identità per l'intero sistema distribuito. Esso concretizza il design pattern Authenticator, accentrando in un unico componente specializzato la responsabilità di verificare le credenziali degli utenti (username e password) e di emettere le relative prove di identità crittografiche.

All'atto della registrazione o del login, le credenziali dell'utente non vengono mai salvate in chiaro nella base di dati PostgreSQL auth_db. La password fornita viene elaborata tramite il pattern Auth-Hash impiegando l'algoritmo di derivazione chiave Scrypt (tramite la classe SCryptUtil), una funzione volutamente memory-hard progettata per neutralizzare attacchi hardware a forza bruta e rainbow table.

In seguito a una verifica con esito positivo delle credenziali, l'AuthService genera e rilascia un Token JWT (Proof of Identity) firmato digitalmente con l'algoritmo simmetrico HMAC-256 e una chiave segreta condivisa. Il token costituisce un'asserzione di identità auto-contenuta: include all'interno dei propri claim l'identificativo utente, lo username, il ruolo assegnato (ADMIN, DOCENTE, STUDENTE) e la scadenza temporale (Expiration Time, con TTL di 24 ore). Trattandosi di una credenziale autonoma e autoverificante, i microservizi a monte possono validare la firma del token conoscendo esclusivamente la chiave segreta condivisa, eliminando la necessità di interpellare l'AuthService per ogni singola richiesta HTTP (comunicazione stateless). L'intero flusso sequenziale di autenticazione, dalla verifica Scrypt alla restituzione del token firmato e al relativo salvataggio della sessione nel portale web, è descritto nel diagramma di sequenza di **Figura 2.1**.



2.1: Diagramma di sequenza del flusso di autenticazione e generazione del JWT via AuthService.

2.2. Server Dati e Logica di Dominio: StudentService

Il microservizio StudentService (in esecuzione sulla porta HTTP 8082) gestisce il dominio applicativo relativo alle anagrafiche degli studenti, ai piani di studio e allo storico dei voti conseguiti su `student_db`.

Esso integra complessi meccanismi di sicurezza e di ottimizzazione prestazionale:

Reference Monitor e Controllo degli Accessi RBAC (PEP/PDP)

Poiché i microservizi backend devono garantire che solo i soggetti autorizzati possano accedere o modificare le risorse di dominio, ogni richiesta HTTP in ingresso allo StudentService viene vagliata in conformità con il pattern Reference Monitor. Il modello teorico viene implementato scindendo la logica di controllo in due componenti distinte:

1. Policy Enforcement Point (PEP): Rappresentato dalla classe `JwtInterceptor` (che implementa `HandlerInterceptor` di Spring), il PEP intercetta indistintamente ogni chiamata HTTP diretta alle rotte protette `/api/**`. Se la richiesta è priva dell'header `Authorization: Bearer <JWT>`, l'intercettore blocca immediatamente l'esecuzione, garantendo che nessuna chiamata non verificata possa raggiungere la logica di business.
2. Policy Decision Point (PDP): Rappresentato dalla classe `JwtValidator`, il PDP riceve il token estratto dal PEP, ne verifica la validità crittografica tramite la firma HMAC-256, ne controlla la scadenza e ne estrae i ruoli utenti.

Sulla base dell'esito del PDP, il sistema applica il modello Role-Based Access Control (RBAC): se l'utente possiede il ruolo richiesto (ad esempio `STUDENTE` per la sola lettura della propria carriera, o `DOCENTE` per l'inserimento esami), la richiesta viene inoltrata al controller di dominio; in caso contrario, viene respinta con codice HTTP 401 Unauthorized o 403 Forbidden.

Caching Trasversale via AspectJ AOP ed Eviction Reattiva

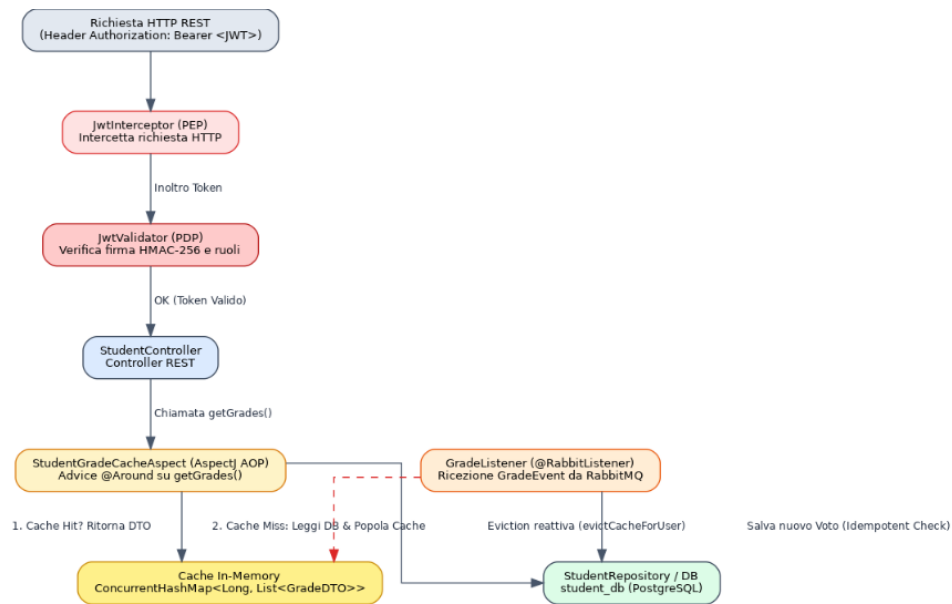
Poiché la consultazione del libretto e della carriera universitaria rappresenta un'operazione ad altissima frequenza di lettura, l'accesso alla base di dati PostgreSQL `student_db` è protetto da un Cache Proxy sviluppato tramite la Programmazione Orientata agli Aspetti (AspectJ AOP). L'aspetto `StudentGradeCacheAspect` intercetta mediante un advice di tipo `@Around` l'esecuzione del metodo `StudentService.getLibrettoByUsername(String)`:

- Cache Hit: Se il libretto dello studente è già presente nella struttura in-memory thread-safe (`ConcurrentHashMap`), l'aspetto intercetta la chiamata e restituisce immediatamente i dati in cache senza accedere al database.
- Cache Miss: Se la chiave non è presente, l'aspetto esegue `JoinPoint.proceed()`, effettua la query JPA su PostgreSQL, memorizza il risultato nella map e lo restituisce al chiamante, eliminando del tutto i problemi di duplicazione e dispersione del codice (code scattering).

Per garantire la consistenza eventuale (Eventual Consistency) dei dati a fronte di nuove verbalizzazioni, il sistema adotta un meccanismo di Eviction Reattiva: non appena lo StudentService consuma un evento AMQP inviato da RabbitMQ, la classe `GradeListener` invoca il metodo statico `StudentGradeCacheAspect.evictCacheForUser(username)`. Tale chiamata rimuove istantaneamente la voce dello studente dalla cache, facendo sì che la successiva richiesta di

2 - Architettura di Dettaglio dei Microservizi

consultazione legge il dato aggiornato da DB. L'architettura interna dello StudentService è illustrata in *Figura 2.2*.



2.2: Architettura interna di StudentService con Reference Monitor (PEP/PDP) e AspectJ AOP Caching.

2.3. Server Dati e Logica di Verbalizzazione: ExamService

Il microservizio ExamService (in esecuzione sulla porta HTTP 8083) si occupa della gestione degli appelli d'esame e delle operazioni di verbalizzazione riservate ai docenti. L'architettura di questo modulo è progettata per supportare modelli di comunicazione asincroni e disaccoppiati.

Disaccoppiamento tramite Remote Facade e DTO

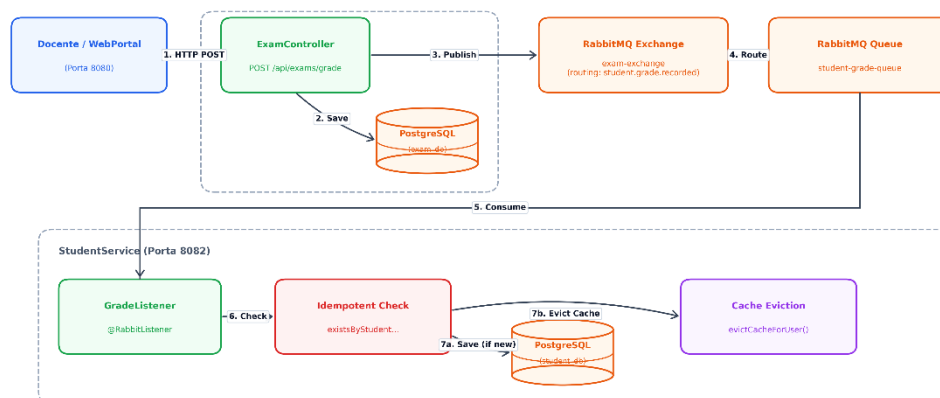
Le API esposte da ExamService utilizzano il pattern Remote Facade per offrire interfacce coarse-grained ottimizzate per le chiamate di rete. Lo scambio dei dati tra il client web e il microservizio avviene esclusivamente mediante Data Transfer Object (DTO) come ExamDTO e GradeDTO. L'uso dei DTO impedisce l'esposizione diretta delle entità di dominio JPA, riduce la dimensione del payload serializzato su rete ed evita problemi di dipendenze cicliche durante la conversione JSON.

Comunicazione Asincrona con Message Broker e Pattern Idempotent Receiver

Quando un docente verbalizza un voto, l'ExamService salva il record nella base di dati exam_db e pubblica in modo asincrono un messaggio di evento GradeEvent sul Message Broker RabbitMQ (utilizzando un Direct Exchange denominato exam-exchange con routing key student.grade.recorded). Questo disaccoppia completamente l'ExamService dallo StudentService, evitando chiamate bloccanti HTTP sincrone durante la verbalizzazione.

Nelle architetture distribuite orientate ai messaggi, le politiche di rete possono causare la ritrasmissione o la consegna duplicata di uno stesso messaggio (at-least-once-delivery). Per evitare che l'eventuale duplicazione del messaggio GradeEvent comporti l'inserimento di voti duplicati nel libretto dello studente, il componente consumatore applica il pattern Idempotent Receiver.

Un'operazione si definisce idempotente quando la sua esecuzione ripetuta produce esattamente il medesimo effetto di una singola esecuzione. Nel nostro sistema, oltre a verificare l'esistenza preventiva della verbalizzazione sul database mediante `existsByStudentAndNomeInsegnamento()`, il listener GradeListener controlla l'esito del voto. Qualora l'esame risulti già superato con esito positivo (voto ≥ 18), eventuali messaggi duplicati pervenuti da RabbitMQ vengono riconosciuti ed ignorati in modo sicuro, garantendo l'idempotenza del sistema. L'intero ciclo di vita del messaggio è rappresentato in *Figura 2.3*.



2.3: Flusso di pubblicazione e consumo dell'evento AMQP GradeEvent su RabbitMQ Broker.

2.4. Server Web e Presentation Layer: web-portal

Il modulo web-portal (in esecuzione sulla porta HTTP 8080) rappresenta l'interfaccia utente centralizzata dell'applicazione. Sviluppato secondo l'architettura Spring Boot MVC e dotato di viste Thymeleaf, esso orchestra la comunicazione con tutti i microservizi sottostanti:

Gestione Ibrida dello Stato di Sessione (Server-Side vs. Client-Side)

La gestione dello stato di sessione nell'architettura Smart University risolve il classico trade-off tra l'esigenza di offrire una navigazione fluida e stateful all'utente web e la necessità di mantenere i microservizi backend rigorosamente stateless. A tal fine, viene applicata una soluzione ibrida:

1. **Server Session State (tra Browser e Web Portal):** L'interazione sulle pagine HTML gestite dal web-portal adotta il pattern Server Session State sfruttando l'oggetto HttpSession di Spring MVC (tracciato lato browser tramite il cookie JSESSIONID). In seguito al login, il portale memorizza nell'HttpSession sia il ruolo dell'utente che il token JWT ottenuto, evitando che l'utente debba autenticarsi nuovamente ad ogni richiesta.
2. **Client Session State (tra Web Portal e Backend):** Nelle comunicazioni REST verso StudentService ed ExamService, si applica il pattern Client Session State. Il portale estrae il JWT dall'HttpSession e lo trasmette nell'header HTTP Authorization: Bearer <JWT>. Poiché il token è auto-contenuto e firmato, i microservizi backend possono convalidarlo autonomamente senza conservare alcuno stato in memoria, rendendo il backend 100% stateless e pronto per la scalabilità orizzontale.

Resilienza e Tolleranza ai Guasti con Circuit Breaker e Fallback

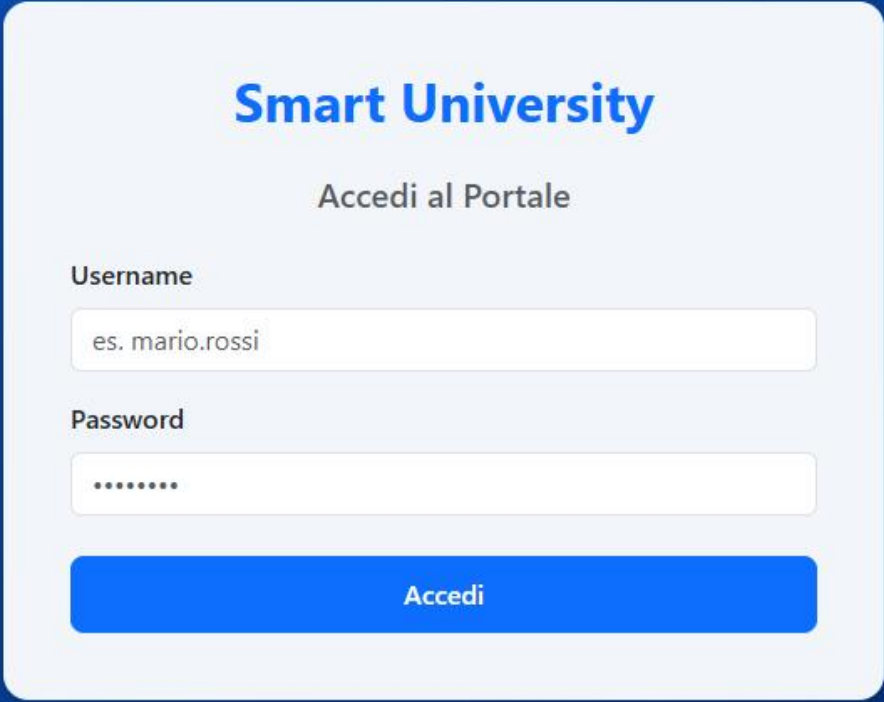
Per evitare che l'eventualità di indisponibilità o rallentamento dello StudentService possa bloccare l'intero portale web, le chiamate HTTP verso l'endpoint `/me/grades` eseguite da StudentWebController sono protette programmaticamente tramite un Circuit Breaker di Resilience4j. Il circuito monitora la frequenza di errore (soglia al 50% su finestra mobile di 5 chiamate); in caso di guasto, la funzione `lambda .recover(...)` della libreria Vavr (*Try.ofSupplier*) intercetta l'eccezione, popola il modello Thymeleaf con un messaggio di *warning* e restituisce una lista vuota. La vista HTML rileva l'avviso e mostra in cima alla pagina il banner di allerta ambra ('Servizio Carriere in Autotutela'), consentendo allo studente di continuare ad iscriversi agli appelli forniti da ExamService senza incorrere in errori HTTP 500.

3. Interfaccia Utente e Descrizione dei Casi d'Uso

L'interfaccia utente di Smart University è progettata per offrire un'esperienza d'uso fluida, reattiva ed intuitiva, adattando dinamicamente le viste HTML presentate in base al ruolo dell'utente autenticato. In questa sezione vengono analizzati i principali flussi applicativi e i relativi casi d'uso.

3.1. Autenticazione e Identificazione Utente

Il flusso operativo ha inizio con la schermata di login accessibile all'URL /login. L'utente inserisce le proprie credenziali (username e password). Il controller del portale inoltra una richiesta POST all'AuthService. In caso di credenziali corrette, il portale riceve il token JWT, lo salva nell'HttpSession dell'utente e reindirizza la pagina verso la dashboard specifica del ruolo (/admin/home, /docente/home o /student/home).



The image shows a login form for 'Smart University'. The form is centered on a dark blue background. It has a light blue header with the text 'Smart University' in bold blue font, followed by 'Accedi al Portale' in a smaller, lighter blue font. Below this, there are two input fields: 'Username' with the placeholder text 'es. mario.rossi' and 'Password' with a masked password '.....'. A large blue button labeled 'Accedi' is positioned below the password field. At the bottom of the form, there is a copyright notice: '© 2026 Università degli Studi di Catania'.

3.1: Interfaccia Utente – Form di Login e autenticazione utenti (login.html).

3.2. Dashboard Amministratore

La console riservata all'amministratore (ruolo ADMIN) permette la gestione centralizzata delle anagrafiche di sistema. L'amministratore può registrare nuovi utenti assegnando i relativi ruoli (DOCENTE o STUDENTE) e creare nuovi insegnamenti associandoli ai docenti titolari.

The screenshot displays the 'Smart University - Pannello Admin' interface. The top navigation bar includes the site name and a settings icon on the left, and the current user 'Amministratore: admin' with a 'Disconnetti' button on the right. The main content area is divided into two side-by-side panels. The left panel, titled 'Registrazione Nuovo Utente', contains a sub-header 'Crea credenziali sicure e profili anagrafici tra i microservizi.' and three input fields: 'Username', 'Password', and 'Ruolo Utente' (a dropdown menu currently showing 'Docente'). A blue button at the bottom reads 'Crea Account e Profilo'. The right panel, titled 'Attivazione Insegnamento', has a sub-header 'Crea un nuovo corso d'insegnamento ufficiale e assegna il docente di ruolo.' and three input fields: 'Codice Corso' (with 'es. INF-01'), 'Nome Insegnamento' (with 'es. Ingegneria dei Sistemi Distribuiti'), and 'Docente Responsabile (Username)' (with 'es. prof.tramontana'). A green button at the bottom reads 'Attiva Insegnamento'.

3.2: Interfaccia Utente – Console dell'Amministratore per la gestione anagrafiche (admin-home.html).

3.3. Dashboard Docente

La vista dedicata ai docenti (ruolo DOCENTE) permette di visualizzare i corsi di cui si è titolari, definire le date delle sessioni d'esame e procedere alla verbalizzazione delle valutazioni. Inserendo la matricola dello studente e la votazione (es. da 18 a 30 e Lode), il docente invia la richiesta al portal web. Come descritto nel Capitolo 2.3, la verbalizzazione viene inoltrata ad ExamService e pubblicata istantaneamente su RabbitMQ, notificando l'avvenuta registrazione senza rallentamenti per il docente.

Smart University

Benvenuto, Prof. **prof.tramontana**
Ruolo: **DOCENTE**
Logout

Pianifica Appello d'Esame

Crea una sessione d'appello d'esame per i tuoi corsi.

Insegnamento

INF-01 - Ingegneria del Software

Data Esame

gg/mm/aaaa

Crea e Pubblica Appello

I Tuoi Appelli Programmati

Gestisci lo stato degli appelli ufficiali.

Insegnamento	Data Esame	Stato	Azione
Ingegneria del Software	2026-10-14	Aperto	Chiudi Appello
Ingegneria dei Sistemi Distribuiti	2026-09-29	Aperto	Chiudi Appello
Ingegneria dei Sistemi Distribuiti	2026-10-13	Aperto	Chiudi Appello
Ingegneria del Software	2026-09-30	Chiuso	Iscrizioni Chiuse

Verbalizzazione Esami (Studenti Prenotati)

Seleziona l'esito per ciascun candidato iscritto agli appelli ufficiali.

Matricola Studente	Insegnamento	Data Appello	Azione Verbalizzazione		
M1234	Ingegneria dei Sistemi Distribuiti	2026-10-13	Promosso	18	☐ Lode Verbalizza
M1234	Ingegneria del Software	2026-10-14	Promosso	30	☒ Lode Verbalizza

3.3: Interfaccia Utente – Dashboard del Docente per creazione appelli e verbalizzazione (docente-home.html).

3.4. Portale Studente

L'interfaccia dello studente (*student-home.html*) organizza le informazioni in tre sezioni dinamiche:

1. **Appelli d'Esame Disponibili:** Il controller *StudentWebController* applica un filtro preventivo (*availableCalls*), mostrando unicamente gli appelli aperti e per i quali lo studente non ha già verbalizzato un esito positivo (voto ≥ 18).
2. **Le Mie Prenotazioni:** Per ogni prenotazione pendente con appello aperto, l'interfaccia offre il pulsante 'Annulla' per revocare l'iscrizione (*/student/cancel-booking*). Lo stato della verbalizzazione viene renderizzato mediante la clausola *th:switch="{booking.stato}"*, garantendo la mutua esclusione visiva tra i badge (verde per 'VERBALIZZATO', rosso per 'RIMANDATO', giallo per 'In attesa').
3. **Libretto Universitario:** Mostra la carriera accademica aggiornata in tempo reale tramite la messaggistica asincrona RabbitMQ ed ottimizzata dall'aspetto di caching AspectJ.

The screenshot displays the 'Smart University' student portal for a user named 'mario.rossi' with the role of 'STUDENTE'. The interface is divided into three main sections:

- Appelli d'Esame Disponibili:** Lists available exams. The table shows two entries for 'Ingegneria dei Sistemi Distribuiti' with dates '2026-09-29' and '2026-10-13', each with a 'Prenotati' button.
- Le Mie Prenotazioni:** Shows the status of bookings. The table lists three bookings:

Insegnamento	Data Esame	Stato Verbalizzazione	Azione
Ingegneria dei Sistemi Distribuiti	2026-10-13	In attesa di verbalizzazione	Annulla
Ingegneria del Software	2026-09-30	Rimandato	-
Ingegneria del Software	2026-10-14	Verbalizzato	-
- Il tuo Libretto Universitario:** Displays the academic record. The table shows one entry:

Insegnamento	Docente Verbalizzante	Esito / Voto
Ingegneria del Software	prof.tramontana	30 e Lode

3.4: Interfaccia Utente – Portale dello Studente per prenotazione appelli e consultazione libretto (*student-home.html*).

4. Trattazione Formale dei Design Pattern Utilizzati

In conformità con il syllabus dell'insegnamento di Ingegneria dei Sistemi Distribuiti, la progettazione di Smart University integra rigorosamente 15 design pattern e scelte architetturali formali. Di seguito viene riportata la tabella sinottica di riepilogo con l'indicazione precisa dei ruoli formali e delle classi Java di riferimento nel codice di progetto.

Pattern	Categoria	Ruolo / Classe Java nel Progetto	Descrizione e Scopo Architetturale
Authenticator	Security	AuthService AuthController	Centralizza l'autenticazione delle credenziali utente e il rilascio di token di identità ufficiali.
Auth-hash (Scrypt)	Security	SCryptUtil	Protegge le password memorizzate nel DB mediante hashing lento resistente a brute-force/GPU.
Token / ProofOfIdentity	Security	JwtService	Rilascia credenziali JWT autonome e firmate HMAC-256 contenenti identità e ruoli del soggetto.
Reference Monitor (PEP/PDP)	Security	JwtInterceptor (PEP) JwtValidator (PDP)	Intercetta tutte le richieste HTTP (PEP) e convalida firme e autorizzazioni tramite il PDP.
Role-Based AC (RBAC)	Security	Role (ADMIN, DOCENTE, STUDENTE)	Associa i permessi ai ruoli degli utenti, controllando l'accesso ai metodi protetti.
Database-per-Service	Architecture	auth_db, student_db, exam_db	Isola le basi di dati PostgreSQL di ciascun microservizio per garantire l'Information Hiding.
Remote Facade	Distribution	StudentController ExamController	Fornisce interfacce coarse-grained per ottimizzare le chiamate REST ed evitare il chattiness.
Data Transfer Object (DTO)	Distribution	GradeDTO GradeEvent	Incapsula i dati trasferiti su rete, evitando l'esposizione diretta delle entità JPA.
Server Session State	Session	HttpSession (in web-portal)	Mantiene lo stato della sessione web lato server sul portale, conservando il token JWT.
Client Session State	Session	Header HTTP Authorization: Bearer	Trasmette il JWT dal portale ai microservizi, mantenendo il backend completamente stateless.
Message Broker (AMQP)	Messaging	RabbitMQ Broker exam-exchange	Disaccoppia temporalmente la verbalizzazione dei voti inviando eventi asincroni a una coda.
Idempotent Receiver	Messaging	GradeListener StudentService.saveGradeFromEvent()	Garantisce che la ricezione duplicata di un evento di voto non generi registrazioni doppi sul DB.
Circuit Breaker	Resilience	CircuitBreaker	Intercetta i guasti nei microservizi backend commutando in OPEN e bloccando chiamate a vuoto.
Fallback	Resilience	.recover(...) (funzione lambda Vavr Try.ofSupplier in StudentWebController)	Fornisce una risposta di cortesia o un valore degradato quando il Circuit Breaker è aperto.
Cache Proxy (AspectJ AOP)	Structural / AOP	StudentGradeCacheAspect @Around advice	Intercetta l'accesso al libretto memorizzando i dati in cache ed eseguendo l'eviction reattiva.

5. Limiti Architetture e Trade-Off

Nonostante l'architettura implementata soddisfi i requisiti di scalabilità e disaccoppiamento, la progettazione ha comportato l'accettazione di alcuni trade-off strutturali, che rappresentano i principali margini di miglioramento del sistema:

- Il problema della Cache In-Memory in un Sistema Distribuito:
L'impiego di *ConcurrentHashMap* con AspectJ all'interno dello *StudentService* garantisce prestazioni eccellenti e bassa latenza in ambiente locale. Tuttavia, in uno scenario di produzione dove lo *StudentService* verrebbe scalato orizzontalmente su molteplici istanze, si genererebbero cache locali isolate. L'eviction reattiva triggered da RabbitMQ svuoterebbe unicamente la cache dell'istanza che ha consumato il messaggio, lasciando dati stantii (stale data) nelle altre. La naturale evoluzione architetturale prevede la migrazione verso un sistema di Distributed Caching (es. Redis o Hazelcast).
- API Gateway e Backend For Frontend (BFF):
Attualmente, il modulo *web-portal* agisce di fatto come un Backend For Frontend (BFF), orchestrando le chiamate REST per conto delle viste Thymeleaf. In un ecosistema più vasto, l'esposizione diretta dei microservizi di dominio (porta 8081, 8082, 8083) andrebbe mascherata adottando formalmente il pattern API Gateway (es. Spring Cloud Gateway), centralizzando così il routing, la gestione dei CORS e il rate-limiting.
- RabbitMQ vs Kafka (Message Broker):
La scelta di RabbitMQ rispetto ad Apache Kafka è stata dettata dalla natura delle operazioni di verbalizzazione. RabbitMQ offre un routing estremamente flessibile e granulare (tramite **Direct Exchange** e routing keys) ed è ottimizzato per pattern di messaggistica transazionale e **fire-and-forget**. Al contrario, Kafka risulta più idoneo per l'event sourcing puro e per lo stream processing massivo, scenari non strettamente necessari per il dominio applicativo in esame.

6. Conclusioni e Sviluppi Futuri

La realizzazione del progetto Smart University ha dimostrato sul campo l'efficacia dell'applicazione combinata dei pattern di progettazione distribuiti, di sicurezza e di resilienza. L'architettura a microservizi disaccoppiata ha permesso di ottenere un'elevata scalabilità modulare, consentendo a ciascun componente di evolvere e riscalarlo in modo indipendente. L'uso combinato di credenziali JWT stateless e del Reference Monitor assicura una protezione perimetrale e granulare delle risorse, mentre il messaging asincrono con RabbitMQ ed il caching AOP AspectJ garantiscono reattività e prestazioni costanti anche in presenza di carichi elevati.

Per quanto concerne gli sviluppi futuri, un'estensione critica riguarda l'Osservabilità (Observability). In un'architettura a microservizi è complesso identificare il modulo responsabile di un'anomalia. L'integrazione di strumenti di Distributed Tracing (es. OpenTelemetry o Zipkin) e l'iniezione di un Correlation ID univoco negli header HTTP e nei payload AMQP, permetterebbe di tracciare l'intera catena di chiamate. Infine, per un deploy production-ready, i microservizi necessiterebbero di essere containerizzati in Docker e orchestrati tramite Kubernetes.

Appendice A: Manuale Operativo e di Deploy

Requisiti di Ambiente ed Installazione

Per eseguire l'intera suite di microservizi in ambiente di sviluppo locale, occorre verificare il soddisfacimento dei seguenti prerequisiti software:

- Java Development Kit (JDK): Versione 17 o 21 (Temurin / Corretto).
- Database Relazionale: PostgreSQL Server 15+ attivo con i tre database `auth_db`, `student_db` ed `exam_db` caricati.
- Message Broker: RabbitMQ Server attivo sulla porta standard AMQP 5672 (gestibile anche tramite contenitore Docker Desktop).
- Ambiente di Build: Apache Maven 3.8+ (oppure l'uso del wrapper `./mvnw` integrato in ciascun modulo).

Inizializzazione del sistema:

Avvio PostgreSQL con creazione automatica dei tre database

```
docker run -d --name postgres-db -p 5432:5432 \  
    -e POSTGRES_USER=admin -e POSTGRES_PASSWORD=adminpassword \  
    -e POSTGRES_MULTIPLE_DATABASES=auth_db,student_db,exam_db  
postgres:15
```

Avvio RabbitMQ Message Broker

```
docker run -d --name rabbitmq -p 5672:5672 -p 15672:15672  
rabbitmq:3-management
```

Guida all'Avvio Sequenziale dei Moduli

Per assicurare che le dipendenze di rete siano correttamente risolte, si consiglia di avviare i moduli aprendo quattro terminali distinti ed eseguendo i comandi nell'ordine seguente:

1. **Avvio dell'AuthService (Porta 8081):**
`cd auth-service && ./mvnw spring-boot:run`
2. **Avvio dello StudentService (Porta 8082):**
`cd student-service && ./mvnw spring-boot:run`
3. **Avvio dell'ExamService (Porta 8083):**
`cd exam-service && ./mvnw spring-boot:run`

4. Avvio del WebPortal (Porta 8080):

```
cd web-portal && ./mvnw spring-boot:run
```

Una volta completata la sequenza di boot, l'applicazione sarà accessibile all'indirizzo web:

<http://localhost:8080/login>.

Lanciando la seguente istruzione da terminale sarà possibile effettuare il primo accesso tramite admin e inserire utenti e corsi così da testare le funzionalità del sistema:

```
curl -X POST http://localhost:8081/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{"username": "admin", "password": "adminpassword",
    "role": "ADMIN"}'
```